An Expert Analysis of Merge Sort, Quick Sort, and Kadane's Algorithm

This report provides a comprehensive and detailed analysis of three fundamental algorithms in computer science: Merge Sort, Quick Sort, and Kadane's Algorithm. The analysis covers their core principles, step-by-step processes, performance characteristics, and practical implementations in C++.

Part I: The Merge Sort Algorithm: A Masterclass in Divide and Conquer

1.1. Foundational Principles: The Divide and Conquer Paradigm

Merge Sort is a quintessential example of the **Divide and Conquer** algorithmic paradigm.[1] This powerful problem-solving technique involves a three-step process:

1. **Divide:** The problem is broken down into smaller, more manageable subproblems that are identical in nature to the original problem.
2. **Conquer:** The subproblems are solved recursively. The base case for the recursion is a subproblem so small that it becomes trivial to solve.
3. **Combine:** The solutions to the subproblems are merged to produce a solution to the original problem.

Merge Sort applies this strategy by recursively dividing an array until it reaches single-element subarrays, which are inherently sorted. The algorithm's efficiency and reliability are a direct result of this systematic, recursive decomposition and reconstruction.

1.2. The Algorithmic Process: A Step-by-Step Breakdown

The Merge Sort process is defined by two primary functions: mergeSort for the recursive division and merge for the combining step.

1.2.1. The "Divide" Phase: Recursive Decomposition

The mergeSort function is responsible for the recursive division of the array. It takes an array and a range of indices (l for left, r for right) and performs the following actions:

- **Base Case:** It first checks if the left index l is less than the right index r. If this condition is false, it means the subarray has one or zero elements, which is the base case of the recursion. A single-element array is considered sorted by definition. The function then returns, allowing the call stack to unwind.
- **Recursive Step:** If the condition l < r is true, the algorithm finds the middle index m of the current subarray. A common and robust way to calculate this is using the formula m = l + (r - l) / 2 to prevent potential integer overflow issues with large arrays. The algorithm then makes two recursive calls: one for the left half of the array (mergeSort(arr, l, m)) and another for the right half (mergeSort(arr, m+1, r)). It is important to note that the second recursive call does not execute until the first one has fully completed its recursive chain, reaching its base case and beginning the merging process.

1.2.2. The "Conquer" Phase: The Merge Operation

The merge function is where the actual sorting and combining occurs. It takes two sorted subarrays (the left and right halves from the recursive calls) and merges them into a single sorted array. The process is as follows:

- **Temporary Arrays:** The merge function first creates temporary auxiliary arrays, often named L and R, to hold the elements of the two subarrays.[2]
- **Comparison and Merging:** It then uses three pointers: one for the left temporary array, one for the right, and one for the original array. It compares the elements at

the current pointers of the temporary arrays and places the smaller element into the original array, incrementing the corresponding pointers.
- **Copying Remaining Elements:** Once one of the temporary arrays is exhausted, the remaining elements from the other array are simply copied back into the original array. This step is necessary because all remaining elements are guaranteed to be larger than any elements already placed.

1.3. A Concrete Walkthrough: Visualizing the Process

Consider the unsorted array ``.

1. The Divide Phase:

- The array is first split into two halves: and.
- The left half, , is further split into and ``.
- The subarray is split into and ``. These are single-element arrays and are considered sorted. The recursion on this branch stops.
- The subarray is split into and ``. These are also single-element arrays. The recursion on this branch also stops.
- The process continues for the right half of the original array, , which is broken down into and and and ``.

2. The Conquer Phase:

- The algorithm now starts merging the single-element subarrays. and are merged to form ``.
- and are merged to form ``.
- The two sorted subarrays, and, are then merged by comparing their elements. The result is the sorted array ``.
- Simultaneously, and are merged into , and and are merged into.
- These two sorted subarrays, and, are merged to form ``.
- Finally, the two large sorted halves, and, are merged to produce the final sorted array ``.

1.4. In-Depth Analysis: Time and Space Complexity

1.4.1. The Recurrence Relation and Its Solution

The time complexity of Merge Sort is best understood through its recurrence relation, which describes the running time for a problem of size n. For Merge Sort, this is expressed as:

$T(n)=2T(n/2)+\Theta(n)$

This equation signifies that the time to sort an array of size n, denoted as T(n), is the sum of the time required to sort two subarrays of size n/2, represented by the 2T(n/2) term, and the linear time Θ(n) required to merge those two sorted subarrays.3 Applying the Master Theorem to this relation yields a consistent time complexity of
$O(nlogn)$.

1.4.2. Deriving Best, Average, and Worst-Case Performance

A unique and highly valuable characteristic of Merge Sort is its consistent performance across all scenarios. The time complexity is O(nlogn) in the best, average, and worst cases.[3] This consistency is not a coincidence; it is a direct result of the algorithm's deterministic structure. The algorithm's design dictates that at every level of recursion, the array is always split into two equal halves. The subsequent merge operation must then compare and place every element, a process that takes linear time,

O(n), regardless of the initial arrangement of the elements. For example, a pre-sorted array requires the exact same number of divisions and merging comparisons as a reverse-sorted array. This rigid, methodical approach ensures that the total number of operations remains constant, making its performance independent of the input order.

1.4.3. Auxiliary Space Requirements

Merge Sort is an **out-of-place** algorithm, which is its primary drawback in memory-constrained environments. Its space complexity is O(n) due to the need for

temporary arrays during the merge step.[6] While the recursive calls themselves also consume space on the function call stack, this overhead is relatively small, at

O(logn), and is dwarfed by the memory required for the temporary arrays.[6] The auxiliary space is a crucial design choice that secures Merge Sort's most significant advantage: its guaranteed

O(nlogn) performance. The use of temporary arrays simplifies the merge operation, allowing it to be completed in linear time and maintaining the algorithm's stability. Therefore, this space is not merely a limitation but a necessary trade-off that underpins the algorithm's robust and predictable nature.

1.5. Key Characteristics and Practical Applications

Merge Sort is a **stable** sorting algorithm, meaning it preserves the relative order of equal elements in the sorted output. This property is highly desirable in situations involving multi-key sorting. Given its guaranteed performance, Merge Sort is often the preferred choice in applications where worst-case time complexity is a critical concern, such as in real-time systems. It is also well-suited for large datasets that may not fit entirely into a system's primary memory, as it forms the basis for external sorting algorithms. However, for very small arrays, its higher constant factors and memory overhead may make it slower than simpler, in-place algorithms like Insertion Sort.

1.6. A Comprehensive C++ Implementation

C++

```
#include <iostream>

// Function to merge two sorted subarrays
void merge(int arr, int l, int m, int r) {
```

```
    int n1 = m - l + 1;
    int n2 = r - m;

    // Create temporary arrays
    int L[n1], R[n2];

    // Copy data to temporary arrays L and R
    for (int i = 0; i < n1; i++) {
        L[i] = arr[l + i];
    }
    for (int j = 0; j < n2; j++) {
        R[j] = arr[m + 1 + j];
    }

    // Merge the temporary arrays back into arr[l..r]
    int i = 0; // Initial index of first subarray
    int j = 0; // Initial index of second subarray
    int k = l; // Initial index of merged subarray

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements of L, if any
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    // Copy the remaining elements of R, if any
    while (j < n2) {
```

```
        arr[k] = R[j];
        j++;
        k++;
    }
}

// Main function that implements Merge Sort
void mergeSort(int arr, int l, int r) {
    if (l < r) {
        // Find the middle point to divide the array into two halves
        int m = l + (r - l) / 2;

        // Sort first and second halves
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);

        // Merge the sorted halves
        merge(arr, l, m, r);
    }
}

// Helper function to print an array
void printArray(int A, int size) {
    for (int i = 0; i < size; i++) {
        std::cout << A[i] << " ";
    }
    std::cout << std::endl;
}

int main() {
    int arr = {12, 11, 13, 5, 6, 7};
    int arr_size = sizeof(arr) / sizeof(arr);

    std::cout << "Given array is \n";
    printArray(arr, arr_size);

    mergeSort(arr, 0, arr_size - 1);

    std::cout << "Sorted array is \n";
```

```
    printArray(arr, arr_size);

    return 0;
}
```

Part II: The Quick Sort Algorithm: Efficiency and Pivot-Driven Partitioning

2.1. Foundational Principles: Partitioning as the Core

Quick Sort, like Merge Sort, is a Divide and Conquer algorithm.[7] However, it reverses the emphasis of the two phases. Instead of performing the hard work during the merge step, Quick Sort's main effort is in the

partitioning step. The algorithm selects a pivot element and rearranges the array so that all elements smaller than the pivot are on one side, and all elements larger are on the other. This partitioning operation places the pivot in its final, sorted position. The algorithm then recursively applies the same process to the two newly created subarrays.[9]

2.2. The Algorithmic Process: Partitioning and Recursive Sorting

The Quick Sort algorithm consists of three main steps:

1. **Choose a Pivot:** A pivot element is selected from the array. This is the most crucial step, as the choice of pivot significantly impacts the algorithm's performance. Common strategies for pivot selection include picking the first element, the last element, a random element, or the median of three elements.[9] A poor pivot choice can lead to highly unbalanced partitions and degrade the algorithm's efficiency.

2. **Partition the Array:** This is the core of the algorithm. The partition function rearranges the elements in the array based on the chosen pivot.[8] After this step, the array is split into two subarrays: one containing elements less than or equal to the pivot, and the other containing elements greater than the pivot. The pivot itself is placed between these two subarrays, effectively putting it in its final sorted position.
3. **Recursively Call:** The quickSort function is then called recursively on the two subarrays, repeating the partitioning process until each subarray consists of a single element, which is the base case for the recursion.[9]

2.3. A Concrete Walkthrough: Partitioning in Action

Consider an unsorted array `` and using the last element, 70, as the pivot.

1. We initialize two pointers, i (index of the smaller element) starting at the beginning of the array minus one, and j (iterator) starting at the beginning of the array.
2. We iterate j from low to high - 1 (the pivot's position).
3. When we encounter arr[j] that is smaller than the pivot (70), we increment i and swap arr[i] with arr[j].
    - The first element, 10, is less than 70, so we increment i to 0 and swap arr with arr. The array remains ``.
    - 80 is greater than 70, so we do nothing.
    - 30 is less than 70, so we increment i to 1 and swap arr (80) with arr (30). The array becomes ``.
    - 90 is greater than 70, so we do nothing.
    - 40 is less than 70, so we increment i to 2 and swap arr (80) with arr (40). The array becomes ``.
    - 50 is less than 70, so we increment i to 3 and swap arr (90) with arr (50). The array becomes ``.
4. After the loop, all elements smaller than 70 are to its left. We then swap the pivot (70) with the element at i+1 (80).
5. The final partitioned array is , and the pivot `70` is now in its correct sorted position. The algorithm then recursively sorts the subarrays and ``.[9]

2.4. In-Depth Analysis: The Impact of Pivot Selection

2.4.1. Average-Case Performance: O(nlogn)

In the average case, when the pivot consistently divides the array into roughly equal halves, Quick Sort's time complexity is O(nlogn).[9] This performance is highly efficient and, in practice, often outperforms other

O(nlogn) algorithms like Merge Sort due to better cache locality and a smaller constant factor.

2.4.2. Worst-Case Performance: O(n2)

The worst-case time complexity of Quick Sort is O(n2).[9] This occurs when the partitioning is highly unbalanced, with one subarray having

n−1 elements and the other having zero.[11] This degenerate case happens when the pivot is consistently chosen as the smallest or largest element in the subarray, such as when the input array is already sorted and the first or last element is always chosen as the pivot.[10] The time complexity in this scenario is described by the recurrence relation

T(N)=T(N−1)+constant×N, which resolves to O(n2). This performance degradation is a direct consequence of a poor pivot choice, which leads to a skewed recursion tree with a depth of O(n) instead of the ideal O(logn).

2.4.3. Space Complexity: The Call Stack's Influence

Quick Sort's space complexity is primarily determined by the depth of its recursion call stack. In the best and average cases, with balanced partitions, the recursion depth is O(logn), leading to a space complexity of O(logn). However, in the worst-case

scenario, where the partitions are completely unbalanced, the recursion depth can reach O(n), resulting in a space complexity of O(n).

2.5. Key Characteristics and Practical Advantages

Quick Sort is an **in-place** sorting algorithm, meaning it requires minimal additional memory, typically O(1), to perform its operations.[9] This makes it more memory-efficient than Merge Sort. However, most standard implementations are

not stable, which means the relative order of equal elements is not guaranteed to be preserved.[12] To mitigate the risk of worst-case performance, a hybrid algorithm called

Introsort is often used in practice.[12] Introsort begins with Quick Sort but switches to Heapsort if the recursion depth becomes too large, thus combining Quick Sort's average-case speed with Heapsort's guaranteed

O(nlogn) worst-case performance. This engineering solution demonstrates a practical approach to overcoming the theoretical limitations of a pure Quick Sort implementation.

2.6. A Comprehensive C++ Implementation

C++

```
#include <iostream>
using namespace std;

// Function to swap two elements
void swap(int* a, int* b) {
    int t = *a;
    *a = *b;
    *b = t;
```

```
}

// This function takes last element as pivot, places the pivot element at its
// correct position in sorted array, and places all smaller elements to the
// left of the pivot and all greater elements to the right.
int partition(int arr, int low, int high) {
    int pivot = arr[high]; // pivot
    int i = (low - 1);     // Index of smaller element

    for (int j = low; j <= high - 1; j++) {
        // If current element is smaller than or equal to pivot
        if (arr[j] <= pivot) {
            i++; // increment index of smaller element
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

// Main function that implements Quick Sort
void quickSort(int arr, int low, int high) {
    if (low < high) {
        // pi is partitioning index, arr[p] is now at right place
        int pi = partition(arr, low, high);

        // Separately sort elements before partition and after partition
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// Function to print an array
void printArray(int arr, int size) {
    int i;
    for (i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}
```

```
int main() {
    int arr = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr);
    quickSort(arr, 0, n - 1);
    cout << "Sorted array: \n";
    printArray(arr, n);
    return 0;
}
```

Part III: Kadane's Algorithm: A Dynamic Programming Solution

3.1. Foundational Principles: Dynamic Programming Meets Greed

Kadane's Algorithm is an elegant and highly efficient algorithm designed to solve the **Maximum Subarray Sum** problem.[13] The problem is to find a contiguous subarray within a one-dimensional array that has the largest possible sum. Kadane's Algorithm is often classified as a dynamic programming or greedy approach because it makes a locally optimal choice at each step, which ultimately leads to a globally optimal solution.[13] The algorithm's core idea is that the maximum subarray sum ending at a given position can be determined by considering the maximum subarray sum of the previous position.

3.2. The Algorithmic Process: The Iterative Approach

The algorithm operates in a single pass through the array, using a constant amount of memory. It relies on two key variables to track its state:

- max_ending_here (or current_sum): This variable stores the maximum sum of a subarray that ends at the current position in the array.
- max_so_far (or max_sum): This variable stores the overall maximum sum found so

far throughout the entire traversal of the array.

The process is as follows [13]:

1. Initialize max_so_far and max_ending_here. For a robust implementation, max_so_far should be initialized to a very small number like INT_MIN or negative infinity, and max_ending_here should be initialized to 0.[14]
2. Iterate through each element of the array.
3. For each element, update max_ending_here by adding the current element to it.
4. If max_ending_here becomes negative, reset it to 0. This is the greedy step, as a negative subarray sum will only decrease the total sum, so it is better to discard it and start a new subarray from the next element.[15]
5. Compare max_so_far with max_ending_here. If max_ending_here is greater, update max_so_far.
6. After iterating through the entire array, max_so_far will hold the maximum subarray sum.

3.3. A Concrete Walkthrough: Navigating Positive and Negative Values

Consider the array [-2, 1, -3, 4, -1, 2, 1, -5, 4].[13]

- Initialize max_so_far = -infinity and max_ending_here = 0.
- **Element -2:** max_ending_here becomes -2. Since it's negative, we reset it to 0. max_so_far remains at -infinity.
- **Element 1:** max_ending_here becomes 1. max_so_far is updated to 1.
- **Element -3:** max_ending_here becomes -2. We reset it to 0. max_so_far remains at 1.
- **Element 4:** max_ending_here becomes 4. max_so_far is updated to 4.
- **Element -1:** max_ending_here becomes 3. max_so_far remains at 4.
- **Element 2:** max_ending_here becomes 5. max_so_far is updated to 5.
- **Element 1:** max_ending_here becomes 6. max_so_far is updated to 6.
- **Element -5:** max_ending_here becomes 1. max_so_far remains at 6.
- **Element 4:** max_ending_here becomes 5. max_so_far remains at 6.

After the loop, the final max_so_far is 6, which is the correct maximum subarray sum for the subarray [4, -1, 2, 1].[13]

3.4. Addressing Edge Cases: The All-Negative Array

A common pitfall in simpler implementations of Kadane's algorithm is its behavior with arrays where all elements are negative.[16] The logic of resetting

max_ending_here to 0 whenever it becomes negative is designed to find the maximum sum of a non-empty subarray, but if an empty subarray with sum 0 is allowed, the algorithm might incorrectly return 0 as the answer.[13] The standard and more robust interpretation of the problem requires finding the single largest (least negative) element in the array as the maximum subarray sum in this case.[14] A robust implementation accounts for this by either initializing

max_so_far to INT_MIN or by using the update logic current_sum = max(num, current_sum + num).[14] This ensures that even when all elements are negative, the algorithm will correctly identify the least negative single element as the maximum sum.

3.5. In-Depth Analysis: The Optimal O(n) Solution

Kadane's algorithm is considered optimal for the Maximum Subarray Sum problem because of its remarkable efficiency. Its time complexity is O(n) as it only requires a single pass through the array, performing a constant number of operations at each step.[13] The algorithm's space complexity is

O(1), as it only uses a few variables to track the current and maximum sums, irrespective of the size of the input array.[14] This makes it an incredibly efficient solution for large datasets.

3.6. A Comprehensive C++ Implementation

The following C++ code implements a robust version of Kadane's algorithm that correctly handles arrays with all negative numbers.

C++

```cpp
#include <iostream>
#include <algorithm> // For std::max
#include <limits>    // For std::numeric_limits

int maxSubarraySum(int arr, int size) {
    // Initialize max_so_far to the smallest possible integer value
    int max_so_far = std::numeric_limits<int>::min();
    int max_ending_here = 0;

    for (int i = 0; i < size; i++) {
        max_ending_here = max_ending_here + arr[i];

        // Update the overall maximum sum if the current sum is greater
        if (max_so_far < max_ending_here) {
            max_so_far = max_ending_here;
        }

        // Reset the current sum to 0 if it becomes negative.
        // This is the key greedy step.
        if (max_ending_here < 0) {
            max_ending_here = 0;
        }
    }

    // A robust check for the case where all elements are negative.
    // If the maximum sum found is 0 (from the reset), but the array
    // contains no non-negative numbers, the max sum should be the largest
    // negative number. A more elegant way to handle this is to initialize
    // max_so_far to the first element and max_ending_here to 0.
    // Let's use an alternative robust approach here:
    if (max_so_far == 0) {
        int max_negative = std::numeric_limits<int>::min();
        for (int i = 0; i < size; ++i) {
            if (arr[i] > max_negative) {
                max_negative = arr[i];
```

```
            }
        }
        return max_negative;
    }

    return max_so_far;
}

int main() {
    int arr1 = {-2, -3, -4, -1, -2, -1, -5, -3};
    int n1 = sizeof(arr1) / sizeof(arr1);
    std::cout << "Maximum subarray sum for all negative array is: " << maxSubarraySum(arr1, n1)
<< std::endl;

    int arr2 = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
    int n2 = sizeof(arr2) / sizeof(arr2);
    std::cout << "Maximum subarray sum for mixed array is: " << maxSubarraySum(arr2, n2) <<
std::endl;

    return 0;
}
```

Part IV: Synthesis and Comparative Analysis

4.1. Merge Sort vs. Quick Sort: A Head-to-Head Comparison

The choice between Merge Sort and Quick Sort is a classic trade-off in algorithm design. While both are highly efficient O(nlogn) sorting algorithms on average, their fundamental differences in approach lead to distinct performance profiles and suitability for different use cases. The key causal difference lies in where the "hard work" of sorting is performed. For Quick Sort, the heavy lifting occurs during the partitioning step, which can be highly efficient on average but leads to quadratic time

in the worst case. Conversely, Merge Sort's hard work is the linear-time merging step, which is performed consistently regardless of the input data, thereby guaranteeing its reliable performance.

Characteristic	Merge Sort	Quick Sort
Time Complexity (Worst)	O(nlogn)	O(n2)
Time Complexity (Best/Avg)	O(nlogn)	O(nlogn)
Space Complexity (Worst)	O(n)	O(n) (Due to call stack)
Space Complexity (Best/Avg)	O(n)	O(logn) (Due to call stack)
Stability	Yes, it is a stable sort.	No, it is generally not stable.
In-place?	No, requires auxiliary space.	Yes, in-place with minimal auxiliary space.
Key Advantage	Guaranteed O(nlogn) performance, reliable for real-time systems, good for external sorting.	Faster average-case performance due to lower constant factors and better cache locality. More memory efficient.
Key Disadvantage	Requires O(n) extra space for temporary arrays.	Vulnerable to worst-case O(n2) performance with poor pivot selection.

4.2. The Broader Context: Algorithmic Paradigms

These algorithms serve as textbook examples of different problem-solving paradigms. Merge Sort and Quick Sort are both built on the **Divide and Conquer** paradigm, demonstrating how a problem can be solved by breaking it down into smaller, recursive subproblems.[1] However, they differ in their implementation of the "conquer" phase: Merge Sort's solution is in the merging step, while Quick Sort's is in the

partitioning. In contrast, Kadane's algorithm showcases the power of a

Dynamic Programming or **Greedy** approach.[13] By making a series of locally optimal decisions at each step (deciding whether to extend the current subarray or start a new one), it arrives at a globally optimal solution in a single, efficient pass.

4.3. Summary and Concluding Remarks

The analysis reveals that no single algorithm is universally superior. Merge Sort offers the distinct advantage of a guaranteed O(nlogn) worst-case time complexity and is a stable sort, making it the ideal choice for applications where predictable performance is non-negotiable, even at the cost of additional memory. Quick Sort, on the other hand, is often the faster choice in practice due to its average-case efficiency and in-place nature, but its susceptibility to worst-case behavior makes robust pivot selection (or using a hybrid like Introsort) a necessity.[12] Finally, Kadane's algorithm provides an optimal and elegantly simple solution to a specific problem with its linear time and constant space complexity, highlighting the efficiency gains of applying a specialized dynamic programming approach. The selection of the right algorithm ultimately depends on a careful consideration of the specific requirements of a given task, including performance guarantees, memory constraints, and the stability of the sort.

Works cited

1. Merge Sort C++: Examples & Implementation - NxtWave, accessed on August 8, 2025, https://www.ccbp.in/blog/articles/merge-sort-cpp
2. Merge Sort - Sort Visualizer, accessed on August 8, 2025, https://sortvisualizer.com/mergesort/
3. Merge Sort algorithm and analysis | Intro to Algorithms Class Notes ..., accessed on August 8, 2025, https://library.fiveable.me/introduction-algorithms/unit-4/merge-sort-algorithm-analysis/study-guide/TiRfUka9L1TjHZSw
4. Code snippet for merge sort algorithm in c++ : r/learnprogramming - Reddit, accessed on August 8, 2025, https://www.reddit.com/r/learnprogramming/comments/exn8vi/code_snippet_for_merge_sort_algorithm_in_c/
5. Merge Sort Algorithm: A Step-by-Step Visualization - YouTube, accessed on August 8, 2025, https://www.youtube.com/watch?v=ho05egqcPI4&pp=0gcJCfwAo7VqN5tD

6. Time and Space Complexity of Merge Sort - Algorithms - Youcademy, accessed on August 8, 2025, https://youcademy.org/merge-sort-time-space-complexity/
7. Quick Sort - GeeksforGeeks, accessed on August 8, 2025, https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/
8. Quick Sort - Sort Visualizer, accessed on August 8, 2025, https://sortvisualizer.com/quicksort/
9. Quick Sort in C++ (Code with Example) | FavTutor, accessed on August 8, 2025, https://favtutor.com/blogs/quick-sort-cpp
10. Time and Space Complexity Analysis of Quick Sort - GeeksforGeeks, accessed on August 8, 2025, https://www.geeksforgeeks.org/dsa/time-and-space-complexity-analysis-of-quick-sort/
11. 6.12. The Quick Sort — Problem Solving with Algorithms and Data Structures, accessed on August 8, 2025, https://runestone.academy/ns/books/published/pythonds/SortSearch/TheQuickSort.html
12. Quicksort - Wikipedia, accessed on August 8, 2025, https://en.wikipedia.org/wiki/Quicksort
13. Kadane's Algorithm: The Ideal Frontier of Subarray Problems, accessed on August 8, 2025, https://www.simplilearn.com/kadanes-algorithm-article
14. Mastering Kadane's Algorithm, accessed on August 8, 2025, https://www.algorithmik.in/guide/data-structures/array/kadane
15. Kadane's Algorithm - Courses, accessed on August 8, 2025, https://neetcode.io/courses/advanced-algorithms
16. Kadane Algorithm Negative Numbers - c++ - Stack Overflow, accessed on August 8, 2025, https://stackoverflow.com/questions/9942228/kadane-algorithm-negative-numbers